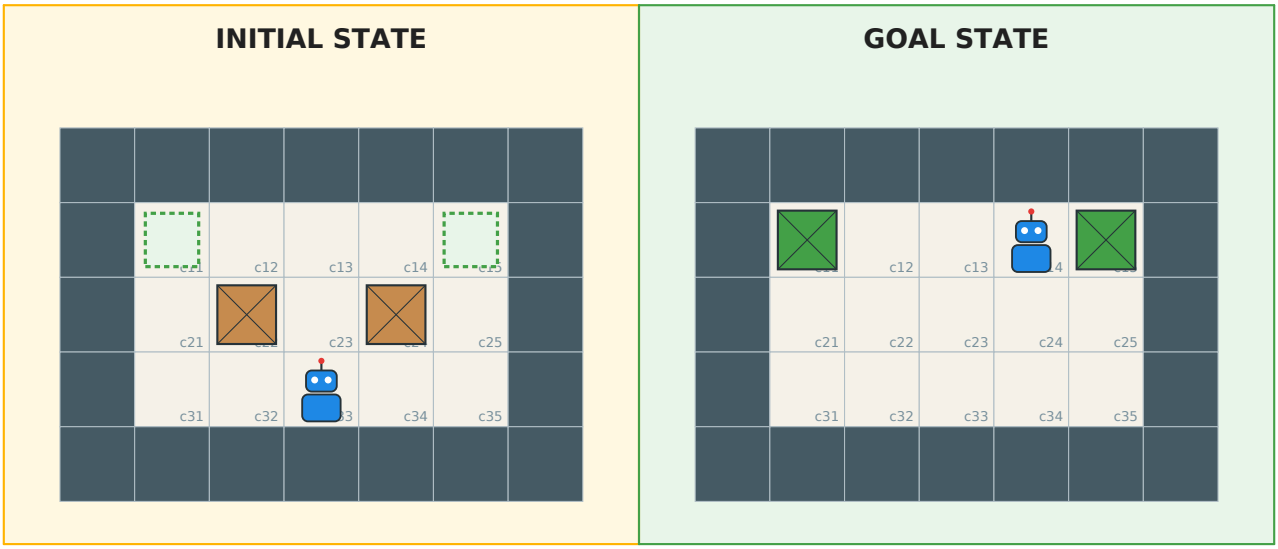
Problem 10: Sokoban

Level: **Medium** · Folder: **10_Sokoban/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

A warehouse robot must push **two boxes** onto the two **goal squares** (dashed green) in a small walled room. The robot can walk onto free floor and can **push** (never pull) a single box one square forward, if the square behind the box is free.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- A 3-argument adjacency relation that includes a **direction**.
- An action with **four parameters** that checks three cells in a straight line.
- **Dead ends**: irreversible actions can make the goal unreachable.
- Goals that ignore irrelevant details (where the robot ends up).

2. Formulating the problem in PDDL

Step 1: Types and objects. cell: the 15 floor cells c11 to c35 (c<row><column>). direction: up, down, left, right.

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(robot-at ?c)`	The robot stands on cell ?c.
`(box-at ?c)`	A box is on cell ?c.
`(clear ?c)`	Cell ?c has neither the robot nor a box on it.
`(adj ?c1 ?c2 ?d)`	Moving from ?c1 in direction ?d leads to ?c2 (static).

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect

move
`?from - cell, ?to - cell, ?d - direction`
Step into a clear neighbouring cell.
	`(robot-at ?from)`
	`(clear ?to)`
	`(adj ?from ?to ?d)`
		`+ (robot-at ?to)`
		`+ (clear ?from)`
		`− (robot-at ?from)`
		`− (clear ?to)`

push
`?r - cell, ?b - cell, ?t - cell, ?d - direction`
Robot at ?r, box at ?b, clear cell ?t, all in a row in direction ?d. The box moves to ?t and the robot moves to ?b.
	`(robot-at ?r)`
	`(box-at ?b)`
	`(clear ?t)`
	`(adj ?r ?b ?d)`
	`(adj ?b ?t ?d)`
		`+ (robot-at ?b)`
		`+ (box-at ?t)`
		`+ (clear ?r)`
		`− (robot-at ?r)`
		`− (box-at ?b)`
		`− (clear ?t)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

INITIAL STATE (59 facts)

```
(robot-at c33)          (adj c13 c12 left)     (adj c32 c22 up)
(box-at c22)            (adj c12 c22 down)     (adj c23 c24 right)
(box-at c24)            (adj c22 c12 up)       (adj c24 c23 left)
(clear c11)             (adj c13 c14 right)    (adj c23 c33 down)
(clear c12)             (adj c14 c13 left)     (adj c33 c23 up)
(clear c13)             (adj c13 c23 down)     (adj c24 c25 right)
(clear c14)             (adj c23 c13 up)       (adj c25 c24 left)
(clear c15)             (adj c14 c15 right)    (adj c24 c34 down)
(clear c21)             (adj c15 c14 left)     (adj c34 c24 up)
(clear c23)             (adj c14 c24 down)     (adj c25 c35 down)
(clear c25)             (adj c24 c14 up)       (adj c35 c25 up)
(clear c31)             (adj c15 c25 down)     (adj c31 c32 right)
(clear c32)             (adj c25 c15 up)       (adj c32 c31 left)
(clear c34)             (adj c21 c22 right)    (adj c32 c33 right)
(clear c35)             (adj c22 c21 left)     (adj c33 c32 left)
(adj c11 c12 right)     (adj c21 c31 down)     (adj c33 c34 right)
(adj c12 c11 left)      (adj c31 c21 up)       (adj c34 c33 left)
(adj c11 c21 down)      (adj c22 c23 right)    (adj c34 c35 right)
(adj c21 c11 up)        (adj c23 c22 left)     (adj c35 c34 left)
(adj c12 c13 right)     (adj c22 c32 down)
```

GOAL

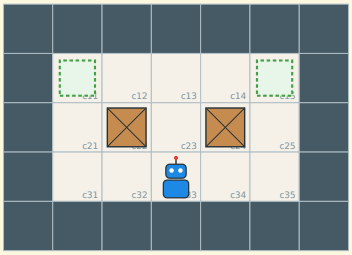
```
(box-at c11)
(box-at c15)
```

Modelling notes

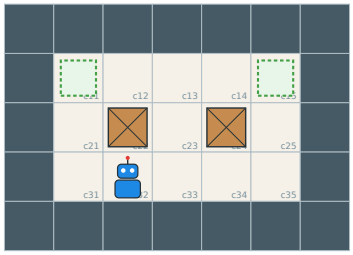
- **Why a direction?** push requires `(adj ?r ?b ?d)` *and* `(adj ?b ?t ?d)` with the **same** ?d. That forces the three cells into a straight line. Without directions the robot could push a box around a corner.
- Walls are not objects. They are cells that do not exist, so no adj fact leads into them.
- Boxes have no names. The goal only says that *some* box is on each goal square. Since the robot's final position is not mentioned, it can be anywhere.
- A box pushed into a corner that is not a goal can never be moved again. This is a dead end that the planner must avoid.

3. Solution plan (11 actions, optimal)

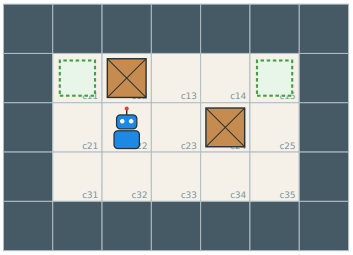
11 actions: each box is pushed up and then sideways into its corner goal. The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



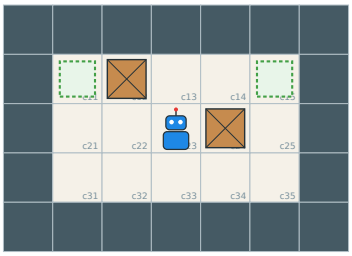
Initial state



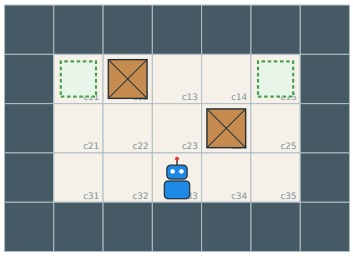
Step 1
`(move c33 c32 left)`



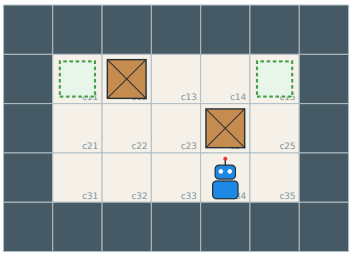
Step 2
`(push c32 c22 c12 up)`



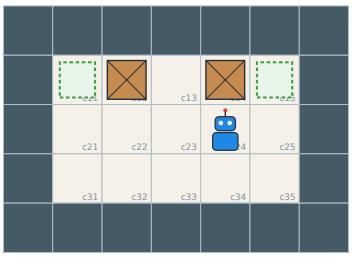
Step 3
`(move c22 c23 right)`



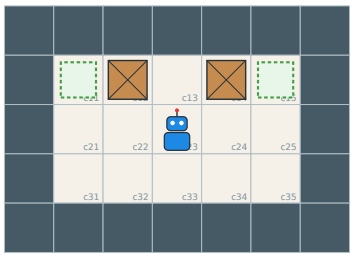
Step 4
`(move c23 c33 down)`



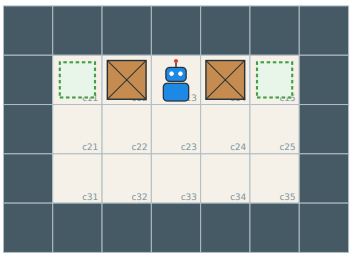
Step 5
`(move c33 c34 right)`



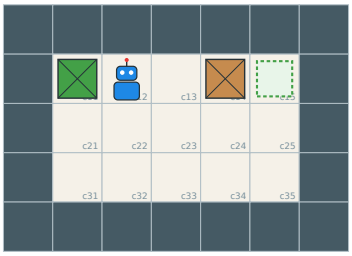
Step 6
`(push c34 c24 c14 up)`



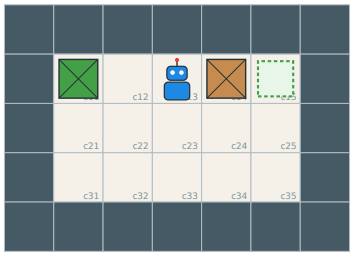
Step 7
`(move c24 c23 left)`



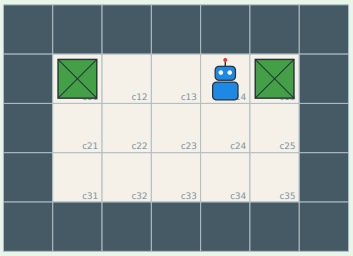
Step 8
`(move c23 c13 up)`



Step 9
`(push c13 c12 c11 left)`



Step 10
`(move c12 c13 right)`



Step 11
`(push c13 c14 c15 right)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. Add a third box and a third goal square, and check that the level is still solvable.
2. Design a level where a wrong first push makes it unsolvable, and observe how the planner avoids it.
3. Add a **pull** action (the robot steps back and drags the box). Does the optimal plan get shorter?

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 10 : Sokoban
;; Level      : Medium
;; A warehouse robot pushes boxes onto goal squares.
;; It can only PUSH (never pull) and only one box at a time.
;; ============================================================
(define (domain sokoban)
  (:requirements :strips :typing)

  (:types cell direction)

  (:predicates
    (robot-at ?c - cell)                    ; robot stands on ?c
    (box-at ?c - cell)                      ; a box is on ?c
    (clear ?c - cell)                       ; ?c holds no robot and no box
    (adj ?c1 - cell ?c2 - cell ?d - direction) ; ?c2 is next to ?c1 in direction ?d
  )

  ;; Walk one step into a free cell
  (:action move
    :parameters (?from - cell ?to - cell ?d - direction)
    :precondition (and (robot-at ?from)
                       (clear ?to)
                       (adj ?from ?to ?d))
    :effect (and (robot-at ?to) (clear ?from)
                 (not (robot-at ?from)) (not (clear ?to)))
  )

  ;; Push the box in front of the robot one step further
  ;;   robot(?r) -> box(?b) -> free(?t)   all in direction ?d
  (:action push
    :parameters (?r - cell ?b - cell ?t - cell ?d - direction)
    :precondition (and (robot-at ?r)
                       (box-at ?b)
                       (clear ?t)
                       (adj ?r ?b ?d)
                       (adj ?b ?t ?d))
    :effect (and (robot-at ?b) (box-at ?t) (clear ?r)
                 (not (robot-at ?r)) (not (box-at ?b)) (not (clear ?t)))
  )
)
```

problem.pddl

```
;; Level map ( # wall, @ robot, $ box, . goal square ):
;;
;;      col: 1 2 3 4 5
;;         # # # # # #
;;   row 1 # .       . #
;;   row 2 #   $   $   #
;;   row 3 #     @     #
;;         # # # # # #
;;
;; Only floor cells are objects; cell names are c<row><col>.
;; Walls are simply cells that do not exist (no adj facts lead there).
(define (problem sokoban-2-boxes)
  (:domain sokoban)

  (:objects
    c11 c12 c13 c14 c15
    c21 c22 c23 c24 c25
    c31 c32 c33 c34 c35 - cell
    up down left right - direction
  )

  (:init
    (robot-at c33)
    (box-at c22) (box-at c24)
    ;; every floor cell without robot or box is clear
    (clear c11) (clear c12) (clear c13) (clear c14) (clear c15) (clear c21)
    (clear c23) (clear c25) (clear c31) (clear c32) (clear c34) (clear c35)
    ;; grid connectivity
    (adj c11 c12 right) (adj c12 c11 left) (adj c11 c21 down) (adj c21 c11 up)
    (adj c12 c13 right) (adj c13 c12 left) (adj c12 c22 down) (adj c22 c12 up)
    (adj c13 c14 right) (adj c14 c13 left) (adj c13 c23 down) (adj c23 c13 up)
    (adj c14 c15 right) (adj c15 c14 left) (adj c14 c24 down) (adj c24 c14 up)
    (adj c15 c25 down) (adj c25 c15 up) (adj c21 c22 right) (adj c22 c21 left)
    (adj c21 c31 down) (adj c31 c21 up) (adj c22 c23 right) (adj c23 c22 left)
    (adj c22 c32 down) (adj c32 c22 up) (adj c23 c24 right) (adj c24 c23 left)
    (adj c23 c33 down) (adj c33 c23 up) (adj c24 c25 right) (adj c25 c24 left)
    (adj c24 c34 down) (adj c34 c24 up) (adj c25 c35 down) (adj c35 c25 up)
    (adj c31 c32 right) (adj c32 c31 left) (adj c32 c33 right) (adj c33 c32 left)
    (adj c33 c34 right) (adj c34 c33 left) (adj c34 c35 right) (adj c35 c34 left)
  )

  (:goal (and (box-at c11) (box-at c15)))
)
```

solution_plan.txt

```
 1. (move c33 c32 left)
 2. (push c32 c22 c12 up)
 3. (move c22 c23 right)
 4. (move c23 c33 down)
 5. (move c33 c34 right)
 6. (push c34 c24 c14 up)
 7. (move c24 c23 left)
 8. (move c23 c13 up)
 9. (push c13 c12 c11 left)
10. (move c12 c13 right)
11. (push c13 c14 c15 right)
```